MODULE 03 · SECTION 1 - DESIGNING DATA PROCESSING SYSTEMS

Encryption and Key Management

Default keys, CMEK, Cloud HSM, Cloud EKM, CSEK and crypto-deletion, and how to tell which one a question wants.

SECTION 1 of 5 EXAM WEIGHT 22% MODULE 3 of 45

admin@sthithapragna.com

Everything is already encrypted at rest. The question is always **who holds the key**.

Envelope encryption: a key in a lockbox

01

Data key

A data encryption key locks each chunk of your data.

02

Key in KMS

A key encryption key wraps the data key. It never leaves the key service.

03

Stored together

Ciphertext sits beside its wrapped data key. Neither is useful alone.

04

To read

The service asks KMS to unwrap the data key, then decrypts.

WHY IT MATTERS

Whoever controls the key encryption key controls the data. That is the lever CMEK hands to you.

Who holds the key: a ladder of control

OPTION	WHERE THE KEY LIVES	CHOOSE IT WHEN
Google default	Google's key service	No stated requirement. The least effort.
CMEK, Cloud KMS	Your key, in Cloud KMS	You must rotate, disable or audit the key yourself.
CMEK, Cloud HSM	Your key, in Google's HSMs	The key must live in certified hardware.
CMEK, Cloud EKM	Outside Google, with a partner	Google must never hold the key material.
CSEK	You send it with each request	Cloud Storage only. You manage everything.
Client-side	Never reaches Google	Google must never see plaintext at all.

THE TIEBREAKER

Climb only as far as the requirement forces you. Every rung adds work and risk.

Default encryption or CMEK?

GOOGLE DEFAULT

Nothing to do

- Encrypted at rest automatically
- Google manages and rotates keys
- No extra cost or setup
- You cannot revoke the key

CMEK

Your key, Google's service

- You set the rotation schedule
- You can disable or destroy it
- Every use of the key is logged
- You must grant and manage access

THE TIEBREAKER

No requirement to control the key means default encryption. 'Control', 'revoke' or 'rotate' means CMEK.

Three things that make CMEK work

- **Location match.** The key ring sits in the same location as the data. A London dataset needs a London key.
- **The service agent** needs Cloud KMS CryptoKey Encrypter/Decrypter on the key. Not you, the service.
- **Defaults.** A BigQuery dataset default key, or a bucket default key, protects new tables and objects.
- **Separate project.** Keys in their own project, so key admins and data admins are different people.

> ON THE EXAM
>
> A CMEK setup that fails with a permission error is nearly always the service agent missing its role.

A CMEK-protected table at Larkspur

```
1  -- Billing data, protected by Larkspur's own key
2  CREATE TABLE `lk-warehouse-prod.curated.bills` (
3    bill_id      STRING,
4    account_id   STRING,
5    amount_gbp   NUMERIC
6  )
7  OPTIONS (
8    kms_key_name =
9      'projects/lk-kms-prod/locations/europe-west2/keyRings/lk-uk/cryptoKeys/billing'
10 );
```

9 The key lives in lk-kms-prod, a separate project for key administration.

9 europe-west2 matches the dataset's London location. A key elsewhere will not work.

BigQuery always protects a new table with the key's primary version.

WORTH KNOWING

The key name never includes a version. BigQuery uses whichever version is primary.

What rotation does, and does not, do

- Rotation creates a **new key version** and makes it primary. New data uses it.
- Existing data is **not re-encrypted**. Older versions stay active to decrypt it.
- To retire an old version, **re-encrypt the data** first. Then you can destroy that version.
- Rotation limits how much data one version protects. It does not rescue a compromised key.

> THE TRAP
>
> If a key is compromised, rotating it is not enough. The old version still decrypts the old data.

Disable, schedule, destroy

STATE	CAN DATA BE READ?	REVERSIBLE?
Enabled	Yes	–
Disabled	No, while disabled	Yes, re-enable it
Scheduled for destruction	No	Yes, 30 days by default
Destroyed	Never again	No. The data is crypto-shredded

THE TIEBREAKER

Disable is the emergency brake. Destroy is the shredder. Questions about suspected misuse want disable first.

Destroy the only key, and the data becomes noise. That is a **deletion strategy**, not an accident.

Cloud HSM or Cloud EKM?

CLOUD HSM

Hardware, run by Google

- Keys in FIPS 140-2 Level 3 HSMs
- Fully managed, no HSM cluster
- Used exactly like any CMEK key
- Google's hardware holds the key

CLOUD EKM

Your key manager, outside Google

- Key material never leaves your partner EKM
- Fortanix, Futurex or Thales
- Every request carries a justification
- If your EKM is down, so is your data

THE TIEBREAKER

'Must be in hardware' is Cloud HSM. 'Keys must never be held by Google' or 'our own HSM' is Cloud EKM.

CSEK, and client-side encryption

- **CSEK is now a Cloud Storage feature.** CSEK for Compute Engine is deprecated. BigQuery never supported it.
- With CSEK you **send the key with every request**. Google does not keep it. Lose it, lose the data.
- **Client-side encryption** encrypts before upload, so Google never sees plaintext.
- The cost: **nothing in Google Cloud can query or process** client-side encrypted data.

> ON THE EXAM
>
> An option offering CSEK for BigQuery is wrong, however attractive it sounds.

One key per customer, with AEAD functions

```sql
-- 1. One keyset per customer, in a tightly restricted dataset
CREATE TABLE secure.customer_keys AS
SELECT customer_id, KEYS.NEW_KEYSET('AEAD_AES_GCM_256') AS keyset
FROM curated.customers;

-- 2. Encrypt personal data with that customer's own keyset
SELECT c.customer_id,
       AEAD.ENCRYPT(k.keyset, c.email, c.customer_id) AS email_enc
FROM curated.customers c JOIN secure.customer_keys k USING (customer_id);

-- 3. Right to erasure: delete one keyset row
DELETE FROM secure.customer_keys WHERE customer_id = 'C-1048812';
```

3 Each customer gets a keyset, generated inside BigQuery. No external tooling.

12 Delete the keyset, and every copy of that customer's ciphertext is unreadable.

HONEST LIMIT

Deleted keysets survive time travel, up to seven days, plus seven days fail-safe. Shorten the keys dataset's window.

Making CMEK compulsory

CONSTRAINT	WHAT IT ENFORCES
`gcp.restrictNonCmekServices`	Listed services, such as storage or BigQuery, refuse new resources without CMEK.
`gcp.restrictCmekCryptoKeyProjects`	CMEK keys may only come from listed projects or folders.
`Both together`	Every new resource uses CMEK, from your approved key project.

REMEMBER

Like all organisation policies here, these apply to new resources. Audit and migrate what already exists.

Autokey: CMEK without the paperwork

- Creates **key rings and keys on demand** when a resource is created, and grants the service agents.
- Keys land in the **same location** as the resource, in a **dedicated key project** for the folder.
- BigQuery gets a **dataset default key**. Buckets, Pub/Sub topics and Dataflow jobs get one key each.
- Pairs naturally with the CMEK organisation policies: compulsory, and effortless.

> ON THE EXAM
>
> 'CMEK everywhere, least operational overhead' is a strong hint toward Autokey.

Encryption in transit, briefly

- Traffic between Google services is **encrypted by default** on Google's network.
- Your clients use **TLS**. For Cloud SQL, the **Auth Proxy** handles it. Module 21.
- On-premises links use **Cloud VPN**, encrypted, or **Interconnect**, private. Module 07.
- Private is not the same as encrypted. Interconnect is private, but add encryption if a rule demands it.

> ON THE EXAM
>
> In-transit questions usually turn on the connection, not the key. Most are answered in later modules.

Phrases that pick the option

THE QUESTION SAYS	THE ANSWER LEANS TO
Control, rotate or revoke our keys	CMEK with Cloud KMS
Keys must be in **hardware**	CMEK with Cloud HSM
Keys **never held by Google**, our own HSM	Cloud EKM
Google must **never see the plaintext**	Client-side encryption
Delete **one customer's** data everywhere	AEAD keysets, crypto-deletion
Every new resource must use CMEK	CMEK organisation policies, perhaps Autokey

ON THE EXAM

No phrase like these at all? Default encryption is already doing the job.

> Larkspur's auditors require that billing data in BigQuery and Cloud Storage be encrypted with keys Larkspur can rotate on its own schedule and revoke immediately, with every key use logged.
>
> Analysts must keep querying the data in BigQuery, and the platform team wants the least operational overhead.

What should Larkspur implement?

A. Keep Google default encryption, and document Google's own rotation schedule for the auditors

B. Use customer-supplied keys, sent with every BigQuery query and Cloud Storage request

C. Encrypt every file client-side with an on-premises key before loading it

D. Use CMEK with Cloud KMS keys in europe-west2, granting each service agent Encrypter/Decrypter

◆ PAUSE AND DECIDE

What should Larkspur implement?

A Google default encryption

Encrypted, yes, but Larkspur cannot set the schedule or revoke the key. The audit asks for both.

B Customer-supplied keys

BigQuery does not support CSEK at all, and sending keys with every request is heavy overhead.

C Client-side encryption

Over-climbs the ladder. Analysts could no longer query the data in BigQuery.

D CMEK with Cloud KMS

Larkspur rotates and revokes, KMS logs every use, and BigQuery queries work as normal.

THE SIGNAL

'Rotate on our schedule' plus 'revoke' is CMEK. 'Must still query' rules out anything above it.

> An engineer creates a key called billing in project lk-kms-prod, in europe-west2. She holds the Cloud KMS Admin role. She then tries to create a CMEK-protected table in a europe-west2 dataset in lk-warehouse-prod.
>
> Table creation fails with a permission error on the key.

What is the most likely fix?

A Grant the engineer Cloud KMS CryptoKey Encrypter/Decrypter on the key

B Grant the BigQuery service account for lk-warehouse-prod Encrypter/Decrypter on the key

C Move the key into lk-warehouse-prod, because keys must be in the same project as the data

D Recreate the key ring in the US multi-region, where BigQuery's key service runs

◆ PAUSE AND DECIDE

What is the most likely fix?

A. Grant the engineer the role

She does not encrypt anything herself. BigQuery does, using its own service account.

B. Grant BigQuery's service account the role

BigQuery wraps and unwraps data keys as its own service account, so that account needs the role.

C. Move the key into the data project

Keys in a separate project are recommended, not forbidden. Project is not the problem.

D. Move the key ring to the US

The opposite of the rule. The key must match the dataset's location, and both are europe-west2.

THE SIGNAL

A permission error on the key during CMEK setup: check the service agent first.

> Larkspur must honour right-to-erasure requests. A customer's personal data appears in dozens of BigQuery tables, and in table snapshots kept for audit.
>
> Finding and deleting every row on each request is impractical. The team wants a native BigQuery approach that makes one customer's data unreadable everywhere at once.

What should the team implement?

A. Encrypt personal columns with AEAD, one keyset per customer, and delete the keyset on request

B. Protect the dataset with one CMEK key, and destroy that key when a request arrives

C. Apply dynamic data masking to the personal columns for that customer

D. Rotate the dataset's CMEK key after each erasure request

◆ PAUSE AND DECIDE

What should the team implement?

A AEAD with a keyset per customer

One deleted row shreds that customer's data in every table and snapshot, and no one else's.

B Destroy one dataset CMEK key

Shreds every customer at once. One erasure request would destroy the whole dataset.

C Dynamic data masking

Hides values from some readers. The data is still stored and readable by others, so nothing is erased.

D Rotate the CMEK key

Rotation re-encrypts nothing and deletes nothing. The old version still decrypts everything.

THE SIGNAL

'One customer, everywhere at once' is per-customer keys. A key shared by everyone can only shred everyone.

> A Cloud Storage bucket of meter exports uses a CMEK key, K1, as its default. The security team believes K1 may have been compromised.
>
> They want the existing objects protected by a fresh key, K1 retired, and a guarantee that no new bucket in the organisation can be created without CMEK.

What should the team do?

A Rotate K1 and keep using the bucket, because rotation replaces the key material

B Destroy K1 immediately, so that nobody, including any attacker, can use it

C Re-encrypt into a bucket using a new key, destroy K1, and enforce restrictNonCmekServices

D Disable K1 and switch the bucket to Google default encryption

◆ PAUSE AND DECIDE

What should the team do?

A **Rotate K1**

Existing objects stay under the old version, which still decrypts them. Nothing is re-protected.

B **Destroy K1 immediately**

Every object still encrypted with K1 becomes unreadable. That is data loss, not remediation.

C **Re-encrypt under a new key, destroy K1, enforce the policy**

Data moves to a fresh key before K1 goes, and the policy covers every future bucket.

D **Disable K1, use default encryption**

Removes CMEK when the requirement was more of it, and does nothing for future buckets.

THE SIGNAL

Compromised key: re-encrypt first, destroy second. 'No new resource without CMEK' adds the policy.

Module 03 in five lines

- Everything is encrypted at rest. The question is **who holds the key**.
- **Control, rotate, revoke** means CMEK. Location must match, and the **service agent** needs the role.
- **Rotation re-encrypts nothing.** A compromised key needs re-encryption, then destruction.
- **Hardware** is Cloud HSM. **Never held by Google** is Cloud EKM. CSEK is Cloud Storage only.
- **Destroy a key to delete data.** Per-customer AEAD keysets shred one customer at a time.

COMING UP

Next: module 04, personal data and Sensitive Data Protection.